# Outras sequências

**REFERÊNCIAS DO RELATÓRIO: NÃO IMPLEMENTADAS NEM TESTADAS**

## Hibbard

$2^k - 1$: 1, 3, 7, 15, 31, ...

**$O(n^{3/2})$**

## Pratt

$2^p 3^q$: 1, 2, 3, 4, 6, 8, 9, 12, ...

**$O(n \log^2 n)$**

## Sedgewick

Sequência específica: 1, 8, 23, 77, 281, ...

**$O(n^{4/3})$**

Limites de pior caso. A escolha dos gaps pode alterar a complexidade.

# Tabela: tempo — entrada aleatória

**SHELL SORT • TEMPO (S)**

| n | Metades | Knuth | Ciura fixo |
|---|---|---|---|
| 10 | 0,000000390 | 0,000000429 | 0,000000438 |
| 100 | 0,000004631 | 0,000004194 | 0,000004305 |
| 1.000 | 0,000072856 | 0,000063018 | 0,000109526 |
| 5.000 | 0,000486612 | 0,000408208 | 0,000425297 |
| 10.000 | 0,001062181 | 0,001042302 | 0,000928731 |
| 50.000 | 0,006674334 | 0,006113780 | 0,005347633 |
| 100.000 | 0,014752876 | 0,013344030 | 0,011474615 |
| 500.000 | 0,090441741 | 0,077902669 | 0,093830671 |
| 1.000.000 | 0,186632804 | 0,164819346 | 0,256181891 |
| 10.000.000 | 2,559364680 | 2,219851804 | 61,100016282 |

Uma medição por combinação. Tempos e contagens não são médias.

# Tabela: tempo — entrada crescente

**SHELL SORT • TEMPO (S)**

| n | Metades | Knuth | Ciura fixo |
|---|---|---|---|
| 10 | 0,000000195 | 0,000000205 | 0,000000232 |
| 100 | 0,000001099 | 0,000000901 | 0,000000953 |
| 1.000 | 0,000015308 | 0,000012286 | 0,000027688 |
| 5.000 | 0,000120905 | 0,000066347 | 0,000076838 |
| 10.000 | 0,000349752 | 0,000141883 | 0,000286576 |
| 50.000 | 0,001540631 | 0,001104427 | 0,000954213 |
| 100.000 | 0,003321665 | 0,001830074 | 0,001811387 |
| 500.000 | 0,018699776 | 0,011814468 | 0,009763198 |
| 1.000.000 | 0,038767026 | 0,024633379 | 0,019712788 |
| 10.000.000 | 0,486784363 | 0,308631088 | 0,198378865 |

Uma medição por combinação. Tempos e contagens não são médias.

# Tabela: tempo — entrada decrescente

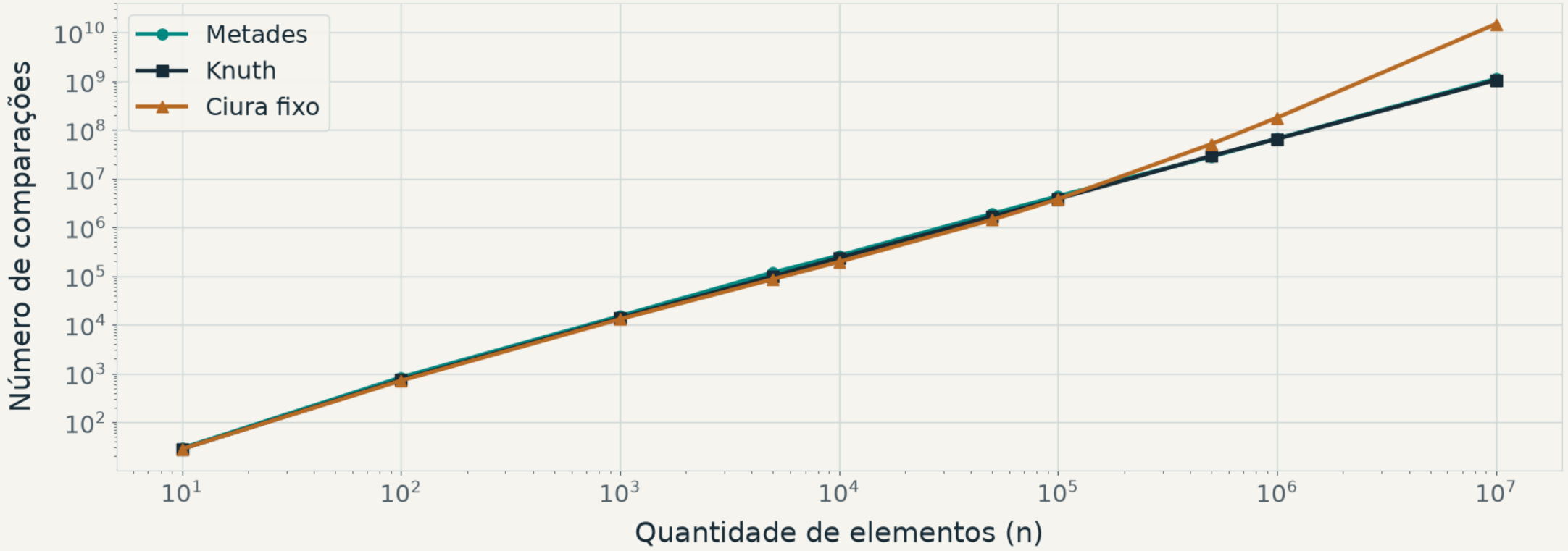
**SHELL SORT • TEMPO (S)**

| n | Metades | Knuth | Ciura fixo |
|---|---|---|---|
| 10 | 0,000000255 | 0,000000281 | 0,000000257 |
| 100 | 0,000001894 | 0,000001529 | 0,000001888 |
| 1.000 | 0,000023160 | 0,000017477 | 0,000021130 |
| 5.000 | 0,000143682 | 0,000110063 | 0,000114380 |
| 10.000 | 0,000315406 | 0,000247542 | 0,000256048 |
| 50.000 | 0,001854750 | 0,001274804 | 0,001665466 |
| 100.000 | 0,003970621 | 0,002514812 | 0,004609766 |
| 500.000 | 0,021537431 | 0,014497618 | 0,081530552 |
| 1.000.000 | 0,047809143 | 0,029639088 | 0,301458130 |
| 10.000.000 | 0,570087394 | 0,366575925 | 136,216577672 |

Uma medição por combinação. Tempos e contagens não são médias.

# Comparações — entrada aleatória

**SHELL SORT • METADES, KNUTH E CIURA FIXO**




Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

# Tabela: comparações — entrada aleatória

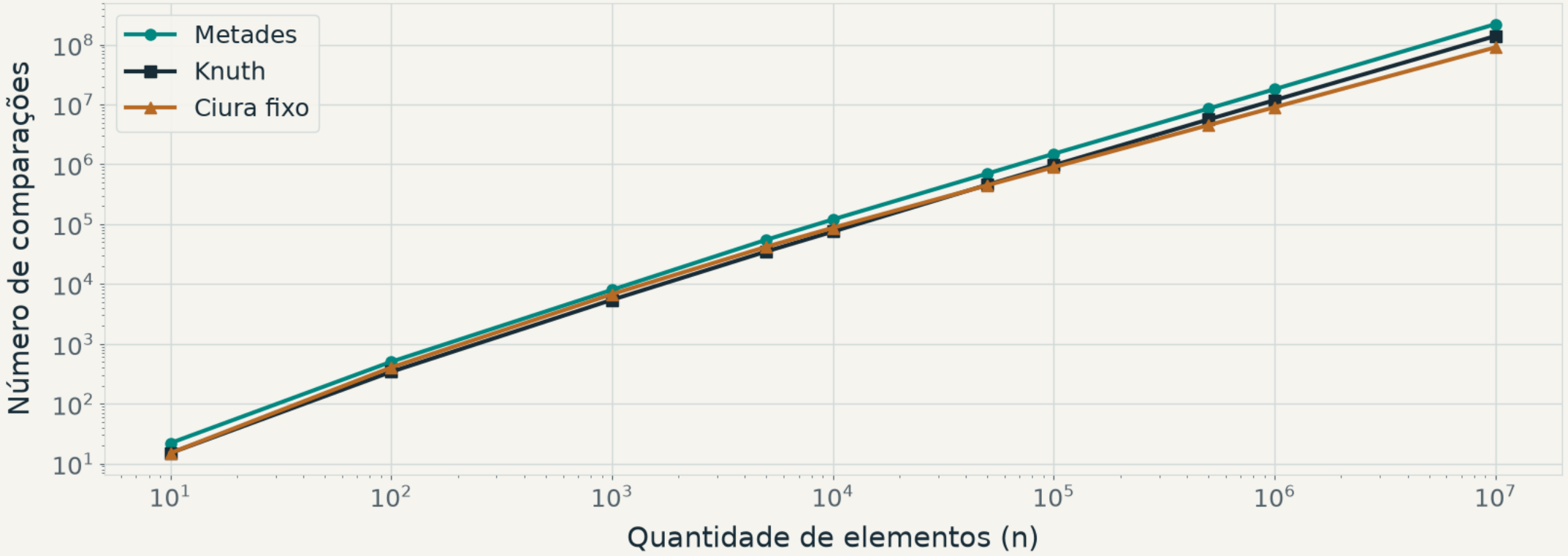
**SHELL SORT • NÚMERO DE COMPARAÇÕES**

| n | Metades | Knuth | Ciura fixo |
|---|---|---|---|
| 10 | 29 | 28 | 28 |
| 100 | 829 | 739 | 711 |
| 1.000 | 15.035 | 13.739 | 13.014 |
| 5.000 | 118.125 | 98.789 | 86.009 |
| 10.000 | 261.534 | 233.623 | 194.969 |
| 50.000 | 1.888.996 | 1.666.757 | 1.419.087 |
| 100.000 | 4.338.950 | 3.830.202 | 3.772.164 |
| 500.000 | 28.072.970 | 29.080.324 | 50.791.358 |
| 1.000.000 | 66.121.589 | 65.318.817 | 177.825.638 |
| 10.000.000 | 1.113.901.611 | 1.047.831.989 | 14.949.158.849 |

Uma medição por combinação. Tempos e contagens não são médias.

# Comparações — entrada crescente

**SHELL SORT • METADES, KNUTH E CIURA FIXO**




Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

# Tabela: comparações — entrada crescente

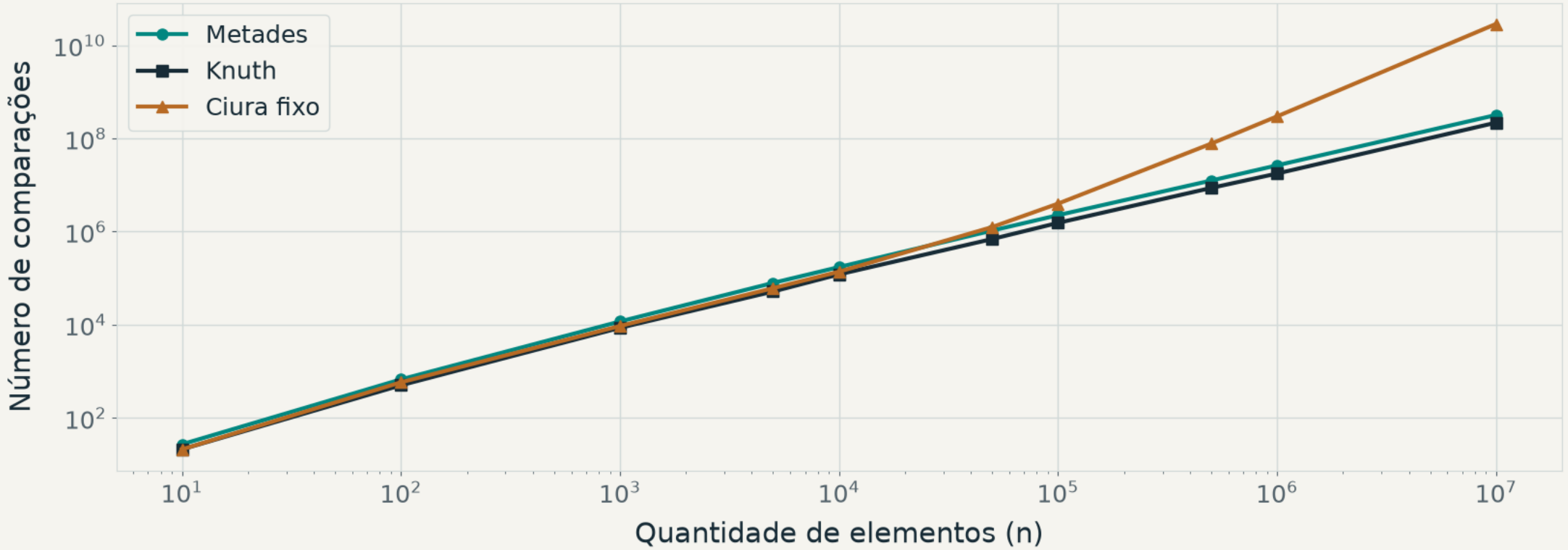
**SHELL SORT • NÚMERO DE COMPARAÇÕES**

| n | Metades | Knuth | Ciura fixo |
|---|---|---|---|
| 10 | 22 | 15 | 15 |
| 100 | 503 | 342 | 405 |
| 1.000 | 8.006 | 5.457 | 6.771 |
| 5.000 | 55.005 | 35.084 | 42.021 |
| 10.000 | 120.005 | 75.243 | 87.021 |
| 50.000 | 700.006 | 455.719 | 447.021 |
| 100.000 | 1.500.006 | 967.146 | 897.021 |
| 500.000 | 8.500.007 | 5.601.426 | 4.497.021 |
| 1.000.000 | 18.000.007 | 11.804.265 | 8.997.021 |
| 10.000.000 | 220.000.008 | 139.238.328 | 89.997.021 |

Uma medição por combinação. Tempos e contagens não são médias.

# Comparações — entrada decrescente

**SHELL SORT • METADES, KNUTH E CIURA FIXO**




Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Entrada decrescente não é, automaticamente, o pior caso matemático.

# Tabela: comparações — entrada decrescente

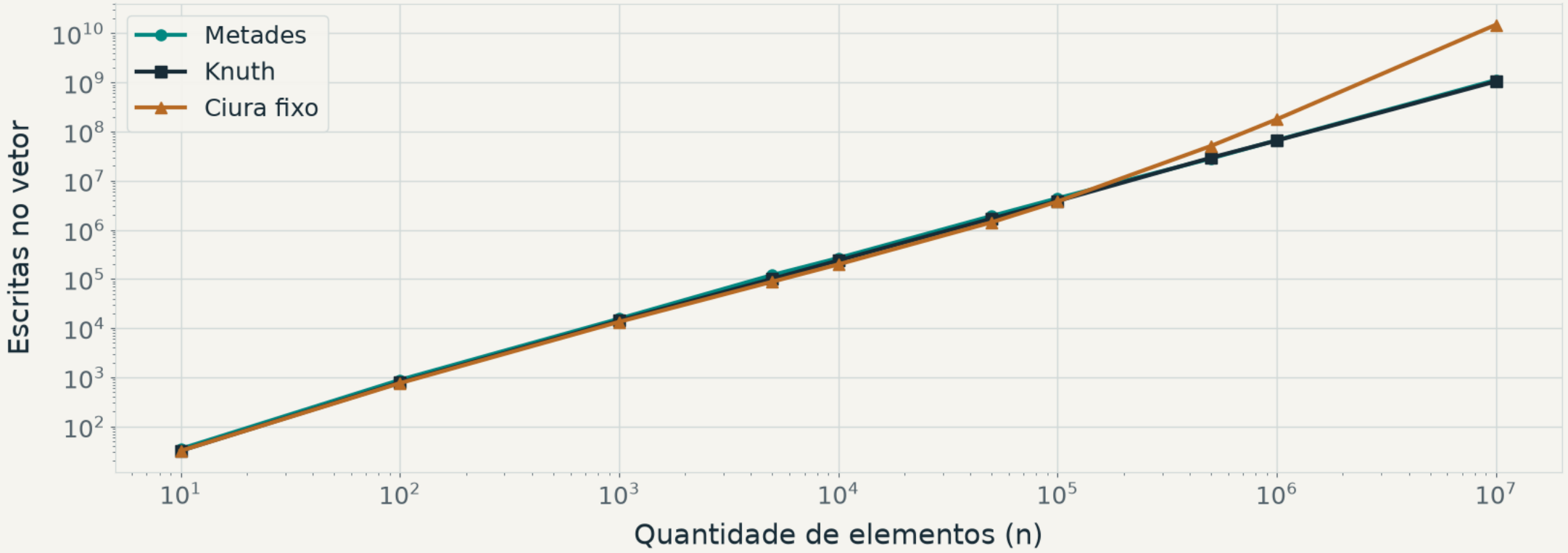
**SHELL SORT • NÚMERO DE COMPARAÇÕES**

| n | Metades | Knuth | Ciura fixo |
|---|---|---|---|
| 10 | 27 | 21 | 21 |
| 100 | 668 | 500 | 579 |
| 1.000 | 11.716 | 8.550 | 9.354 |
| 5.000 | 78.798 | 51.491 | 60.712 |
| 10.000 | 172.578 | 120.190 | 137.075 |
| 50.000 | 1.047.305 | 689.218 | 1.246.058 |
| 100.000 | 2.244.585 | 1.533.494 | 3.968.350 |
| 500.000 | 12.428.778 | 8.622.570 | 76.944.995 |
| 1.000.000 | 26.357.530 | 17.578.626 | 296.380.356 |
| 10.000.000 | 317.626.219 | 217.670.525 | 28.681.792.543 |

Uma medição por combinação. Tempos e contagens não são médias.

# Movimentações — entrada aleatória

**SHELL SORT • METADES, KNUTH E CIURA FIXO**




Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Escritas no vetor, incluindo a gravação final do temporário.

# Tabela: movimentações — entrada aleatória

**SHELL SORT • ESCRITAS NO VETOR**

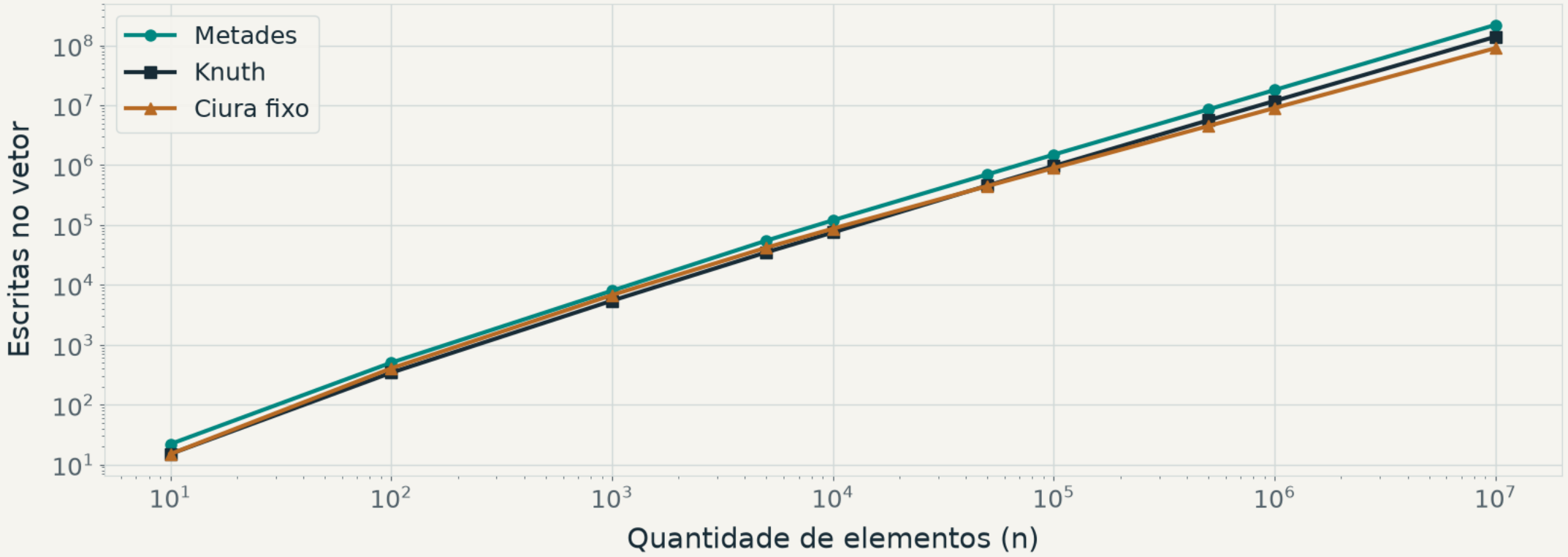
| n | Metades | Knuth | Ciura fixo |
|---|---|---|---|
| 10 | 35 | 32 | 32 |
| 100 | 885 | 784 | 761 |
| 1.000 | 15.547 | 14.174 | 13.462 |
| 5.000 | 120.673 | 100.966 | 88.255 |
| 10.000 | 266.629 | 237.985 | 198.441 |
| 50.000 | 1.914.433 | 1.689.682 | 1.425.324 |
| 100.000 | 4.389.591 | 3.871.711 | 3.779.446 |
| 500.000 | 28.325.796 | 29.317.469 | 50.801.744 |
| 1.000.000 | 66.623.940 | 65.727.764 | 177.837.084 |
| 10.000.000 | 1.118.919.386 | 1.051.977.554 | 14.949.174.355 |

Uma medição por combinação. Tempos e contagens não são médias.

Movimentações incluem a gravação final do temporário.

# Movimentações — entrada crescente

**SHELL SORT • METADES, KNUTH E CIURA FIXO**




Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Escritas no vetor, incluindo a gravação final do temporário.

# Tabela: movimentações — entrada crescente

**SHELL SORT • ESCRITAS NO VETOR**

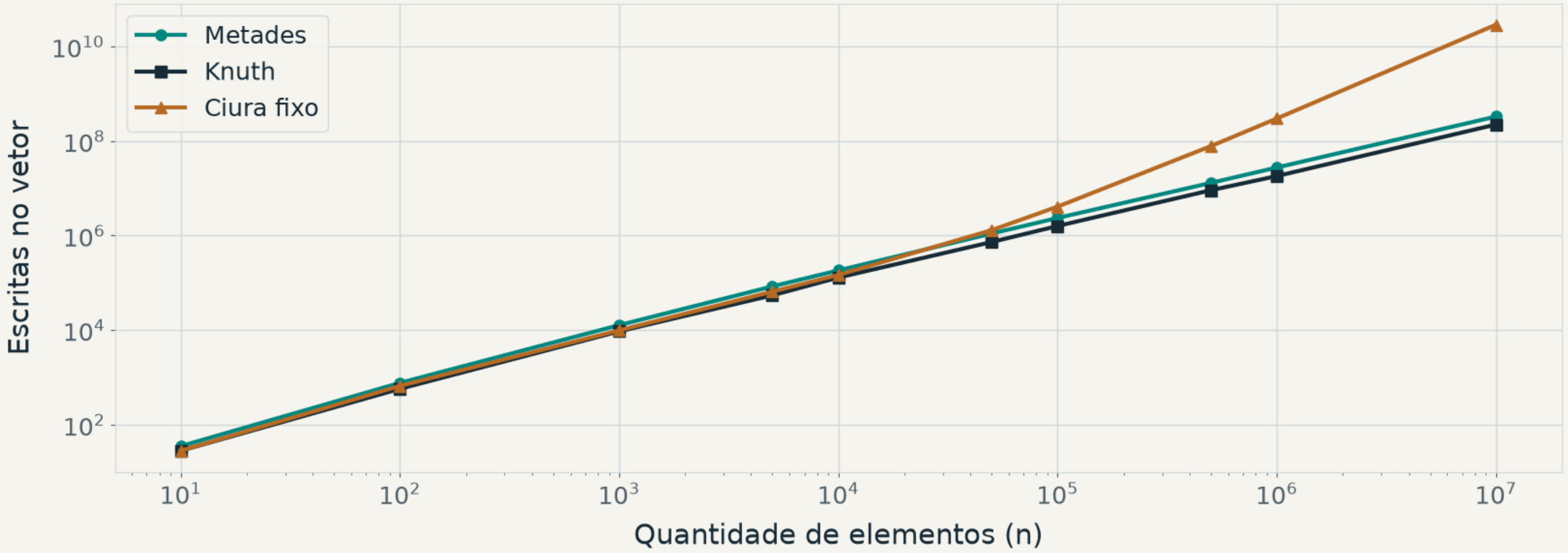
| n | Metades | Knuth | Ciura fixo |
|---|---|---|---|
| 10 | 22 | 15 | 15 |
| 100 | 503 | 342 | 405 |
| 1.000 | 8.006 | 5.457 | 6.771 |
| 5.000 | 55.005 | 35.084 | 42.021 |
| 10.000 | 120.005 | 75.243 | 87.021 |
| 50.000 | 700.006 | 455.719 | 447.021 |
| 100.000 | 1.500.006 | 967.146 | 897.021 |
| 500.000 | 8.500.007 | 5.601.426 | 4.497.021 |
| 1.000.000 | 18.000.007 | 11.804.265 | 8.997.021 |
| 10.000.000 | 220.000.008 | 139.238.328 | 89.997.021 |

Uma medição por combinação. Tempos e contagens não são médias.

Movimentações incluem a gravação final do temporário.

# Movimentações — entrada decrescente

**SHELL SORT • METADES, KNUTH E CIURA FIXO**




Escala logarítmica nos dois eixos. Uma medição por combinação, sem médias.

Escritas no vetor, incluindo a gravação final do temporário.

# Tabela: movimentações — entrada decrescente

**SHELL SORT • ESCRITAS NO VETOR**

| n | Metades | Knuth | Ciura fixo |
|---|---|---|---|
| 10 | 35 | 28 | 28 |
| 100 | 763 | 572 | 659 |
| 1.000 | 12.706 | 9.377 | 9.861 |
| 5.000 | 83.785 | 54.296 | 65.105 |
| 10.000 | 182.565 | 128.947 | 146.331 |
| 50.000 | 1.097.286 | 726.487 | 1.294.815 |
| 100.000 | 2.344.566 | 1.586.800 | 4.067.357 |
| 500.000 | 12.928.759 | 9.053.230 | 77.444.251 |
| 1.000.000 | 27.357.511 | 18.049.649 | 297.379.027 |
| 10.000.000 | 327.626.184 | 221.971.074 | 28.691.791.291 |

Uma medição por combinação. Tempos e contagens não são médias.

Movimentações incluem a gravação final do temporário.

# Selection Sort

História, código em C e análise de desempenho




Projeto e Análise de Algoritmos | 2026

Iniciar

# Como surgiu o Selection Sort?

## A ideia é simples

Procurar o menor valor, colocá-lo no início e repetir com os restantes.

## Nos anos 1950

Pesquisadores já estudavam formas de ordenar dados nos primeiros computadores.

**É dessa época que vêm alguns dos registros mais antigos de métodos de seleção.**

Base histórica: Rochester (2021) e estudo de Owen Astrachan, Duke (2003).

# 1956: um dos primeiros registros

**1956**

## Howard B. Demuth

Estudou formas de ordenar dados
na Universidade de Stanford.

Em seu doutorado, ele descreveu uma técnica que depois foi relacionada ao Selection Sort.

É um registro antigo, mas não prova quem inventou o algoritmo.

Fonte principal: Alan Beadle, Historical Origins of Data Structures and Algorithms (2021), p. 24.

# Outros estudos em 1956 e 1958

**1956**

## Edward H. Friend

Publicou um estudo sobre como ordenar dados em computadores.

**1958**

## D. A. Bell

Também apresentou métodos para ordenar informações.

Esses métodos eram parecidos, mas não exatamente iguais à versão atual.

Análise histórica: Owen Astrachan (Duke, 2003), seção 2.2.

# Como passou a ser ensinado

**1973** Knuth publica um livro importante sobre algoritmos de ordenação.

**1998** Uma edição do livro traz um capítulo sobre ordenação por seleção.

**2000** O MIT apresenta Selection Sort em suas aulas de programação.

Fontes: bibliografia de Knuth, editora Addison-Wesley / InformIT e MIT OpenCourseWare.

# Como funciona

A cada rodada, o algoritmo encontra o menor valor da parte restante e o troca com a primeira posição livre.




O processo continua até todas as posições ficarem definidas.

Ver exemplo

# A ideia central do Selection Sort

## Selecionar o menor restante e colocá-lo na próxima posição.

### Busca

Compara os valores e guarda a posição do menor.

### Troca

Move o menor para o início da parte ainda não ordenada.

A parte ordenada permanece pronta nas rodadas seguintes.

Rever exemplo

Código em C

# Implementação real do trabalho

**O PROGRAMA USA ARQUIVOS DE ENTRADA, NÃO UM VETOR DIGITADO**

**1** **Ler dados**

Arquivos .txt com n inteiros.

**2** **Ordenar**

selection_sort() em C.

**3** **Registrar**

Comparações, movimentos e tempo.

## 30 execuções = 10 tamanhos × 3 tipos de entrada

Aleatória • Crescente • Decrescente

Os arquivos de teste devem existir antes de iniciar a bateria; este código apenas os lê.

**Código real**

# Código em C: Selection Sort

**TRECHO RETIRADO DA FUNÇÃO selection_sort() ENVIADA**

```
static void selection_sort(int *v, size_t n) {
    for (size_t i = 0; i + 1 < n; ++i) {
        size_t menor = i;
        for (size_t j = i + 1; j < n; ++j) {
            comparacoes++;
            if (v[j] < v[menor]) menor = j;
        }
        if (menor != i) {
            int tmp = v[i];
            v[i] = v[menor];
            v[menor] = tmp;
            movimentacoes += 2;
        }
    }
}
```

### i e j

i define a posição; j percorre o restante.

### comparacoes++

Soma 1 ao comparar valores.

### movimentacoes += 2

Troca efetiva = 2 escritas no vetor.

# Como o programa executa cada teste

**FUNÇÕES PRESENTES NO ARQUIVO selection_sort(1).c**

| | |
|---|---|
| **main()** | Localiza a pasta, verifica os 30 arquivos e coordena os testes. |
| **carregar()** | Lê exatamente n inteiros do arquivo .txt. |
| **executar()** | Aloca memória, ordena, valida e preenche Resultado. |
| **segundos()** | Calcula a diferença entre início e fim em segundos. |
| **iniciar_relatorios()** | Cria arquivos selection.csv e selection.txt. |
| **finalizar_relatorios()** | Registra a duração total do programa e fecha os relatórios. |

O vetor é liberado com free() após cada execução, inclusive nos caminhos de erro.

# Bateria de testes: 30 combinações

**10 TAMANHOS DE VETOR E TRÊS ORGANIZAÇÕES DE ENTRADA**

| ALEATÓRIO | CRESCENTE | DECRESCENTE |
|---|---|---|
| Números em ordem não organizada | Vetor já em ordem | Valores inicialmente invertidos |

## Tamanhos utilizados (n)

| | | | | |
|---|---|---|---|---|
| **10** | **100** | **1.000** | **5.000** | **10.000** |
| **50.000** | **100.000** | **500.000** | **1.000.000** | **10.000.000** |

1 medição por combinação (não são médias). O código exige os 30 arquivos previamente.

# O que é contabilizado?

**COMPARAÇÕES, TROCAS E MOVIMENTAÇÕES NÃO SÃO A MESMA COISA**

## Comparações

comparacoes++ é executado no laço interno, independentemente de ocorrer troca.

## Movimentações

Uma troca entre posições diferentes soma exatamente 2 escritas no vetor.

## Exemplo com 5 elementos

[64, 25, 12, 22, 11] **10 comparações** **3 trocas = 6 movimentações**

Se o vetor já estiver ordenado: 10 comparações, 0 trocas e 0 movimentações.

# Crescimento quadrático na prática

**AS COMPARAÇÕES CRESCEM MUITO MAIS RÁPIDO DO QUE n**

| | |
|---|---|
| **n = 5** | **10** comparações |
| **n = 100** | **4.950** comparações |
| **n = 1.000** | **499.500** comparações |
| **n = 10.000** | **49.995.000** comparações |

Multiplicar n por 10 aumenta a contagem de comparações em aproximadamente 100 vezes.

# Como o desempenho foi medido

**RELATÓRIOS CSV/TXT E VALIDAÇÃO APÓS CADA ORDENAÇÃO**

**1** **Cronômetro**

clock_gettime(): relógio monotônico

**2** **Trecho medido**

Somente selection_sort()

**3** **Conferência**

Vetor final em ordem crescente

**selection.csv** **selection.txt**

Tipo de entrada, n, tempo, comparações, movimentações e validação.

Os resultados publicados são medições individuais, não médias de várias execuções.

**Resultados**

# Resultados registrados no relatório

**TESTE DE 10 MILHÕES DE ELEMENTOS ALEATÓRIOS**

**Selection Sort**

**107.310,582 s**

aproximadamente 29 h e 49 min

**Para contextualizar**

**Shell Sort por metades**

**2,559 s**

**Selection realiza 49.999.995.000.000 comparações com n = 10.000.000.**

Valores informados no PDF do trabalho; não foram executados novamente nesta atualização.

# Conclusões: parte Selection Sort

SÍNTESE DA IMPLEMENTAÇÃO E DA ANÁLISE

1

## A quantidade de comparações é fixa

C(n) = n(n − 1) / 2, em qualquer ordem inicial.

2

## Movimentações dependem do vetor

Uma troca efetiva gera duas escritas no vetor.

3

## Em vetores grandes, o custo domina

$O(n^2)$ e $\Omega(n^2)$ resultam em $\Theta(n^2)$.

Início

# Tabela de tempos — entrada aleatória

**SELECTION × SHELL POR METADES • TEMPO EM SEGUNDOS**

| n | Selection Sort | Shell por metades |
|---|---|---|
| 10 | 0,000000215 | 0,000000390 |
| 100 | 0,000010901 | 0,000004631 |
| 1.000 | 0,001083199 | 0,000072856 |
| 5.000 | 0,027786688 | 0,000486612 |
| 10.000 | 0,107050143 | 0,001062181 |
| 50.000 | 2,695467939 | 0,006674334 |
| 100.000 | 10,777727863 | 0,014752876 |
| 500.000 | 267,950135405 | 0,090441741 |
| 1.000.000 | 1062,470905078 | 0,186632804 |
| 10.000.000 | 107310,581887928 | 2,559364680 |

Uma medição por combinação. Tempos e contagens não são médias.

# Tabela de tempos — entrada crescente

**SELECTION × SHELL POR METADES • TEMPO EM SEGUNDOS**

| n | Selection Sort | Shell por metades |
|---|---|---|
| 10 | 0,000000164 | 0,000000195 |
| 100 | 0,000007785 | 0,000001099 |
| 1.000 | 0,001075995 | 0,000015308 |
| 5.000 | 0,027275822 | 0,000120905 |
| 10.000 | 0,107220236 | 0,000349752 |
| 50.000 | 2,699136068 | 0,001540631 |
| 100.000 | 10,789414413 | 0,003321665 |
| 500.000 | 265,359122352 | 0,018699776 |
| 1.000.000 | 1061,405770046 | 0,038767026 |
| 10.000.000 | 105912,839734180 | 0,486784363 |

Uma medição por combinação. Tempos e contagens não são médias.

# Tabela de tempos — entrada decrescente

**SELECTION × SHELL POR METADES • TEMPO EM SEGUNDOS**

| n | Selection Sort | Shell por metades |
|---|---|---|
| 10 | 0,000000192 | 0,000000255 |
| 100 | 0,000009186 | 0,000001894 |
| 1.000 | 0,001062472 | 0,000023160 |
| 5.000 | 0,027273864 | 0,000143682 |
| 10.000 | 0,107271368 | 0,000315406 |
| 50.000 | 2,686454269 | 0,001854750 |
| 100.000 | 10,800248744 | 0,003970621 |
| 500.000 | 265,425003356 | 0,021537431 |
| 1.000.000 | 1061,469464228 | 0,047809143 |
| 10.000.000 | 105923,688492376 | 0,570087394 |

Uma medição por combinação. Tempos e contagens não são médias.